

PROGRAMME DE FORMATION · VERSION 3 · 2026-09-13

Design Patterns : Conception avec les Design Patterns

Choisissez et appliquez les design patterns à partir de problèmes de conception observables. Vous comparez les compromis et conduisez un refactoring progressif protégé par des tests de comportement.

3 jours · 21 heures

Intermédiaire · Hybride

2 440 € HT

Tarif indicatif, dates, disponibilité et modalités contractuelles à confirmer avec le centre.

À qui s'adresse cette formation ?

Développeurs et concepteurs logiciels souhaitant améliorer la conception objet et conduire des refactorings.

Prérequis

Les participants à cette formation Design Patterns doivent disposer d'une solide expérience en programmation orientée objet (Java, C++...). Une confrontation préalable à l'un de ces langages vous aura permis d'assimiler les problématiques de développement et de conception les plus courantes. Les bénéfices des design patterns en seront d'autant plus appréciés.

Objectifs pédagogiques

- Connaître l'origine, la philosophie et l'utilité des Design Patterns
- Connaître les trois grandes familles de patterns et les principaux patrons au sein de chacune de ces familles
- Savoir identifier, pour un problème donné, le patron de conception le plus à même de solutionner le problème
- Savoir appliquer de manière concrète le design pattern ainsi identifié

Le cas fil rouge

Refactorer un module de tarification évolutif en choisissant des design patterns seulement lorsqu'ils simplifient un problème réel.

Préparation

Programmation objet dans un langage maîtrisé ; petit projet avec tests de comportement fournis pour le laboratoire.

Progression détaillée

Répartition pédagogique proposée sur une base de 7 heures par jour, pauses exclues. À ajuster avec le formateur au positionnement initial. Les approfondissements « Pour aller plus loin » sont sélectionnés par le formateur selon le positionnement et le temps disponible ; ils ne constituent pas des heures supplémentaires garanties.

Jour 1 · 7 heures

1. Reconnaître un problème de conception

210 minutes · apports, exercice et retour

- Repérer couplage, duplication et responsabilités mêlées.
- Distinguer patron, principe de conception et recette automatique.
- Formuler les variations que le logiciel doit supporter.
- Repérer couplage, responsabilités dispersées et points de variation avant de choisir un pattern.

NOTIONS ET MÉTHODES EN DÉTAIL

- Repérer duplication, couplage, changement divergent et responsabilité trop large.
- Relier principe de substitution, inversion de dépendance et contrat testé.
- Distinguer besoin de variation réel et complexité ajoutée par anticipation.

Atelier prévu

Analyser un module de tarification et définir trois changements futurs avant de choisir une solution.

Livrable attendu

Diagnostic de conception et critères de choix.

Vérification en atelier

Le diagnostic formule le problème avant de nommer un pattern.

Pour aller plus loin · selon positionnement et temps disponible

Justifier l'absence de pattern quand une solution simple suffit.

2. Séparer la création des objets

210 minutes · apports, exercice et retour

- Comparer Factory Method, Abstract Factory et Builder.
- Encapsuler construction complexe et dépendances variables.
- Discuter limites des singletons et de l'état global.
- Préserver les invariants dès la construction, injecter les dépendances et distinguer configuration d'objet et sélection de sa famille concrète.

NOTIONS ET MÉTHODES EN DÉTAIL

- Comparer Factory Method, Abstract Factory et Builder selon variantes et construction progressive.
- Séparer décision de création et code consommateur avec une interface stable.
- Examiner les risques d'état global et de dépendances cachées d'un Singleton.

Atelier prévu

Remplacer une construction dispersée d'objets par une fabrique puis tester une nouvelle famille de tarifs.

Livrable attendu

Implémentation de création et tests d'invariants.

Vérification en atelier

Le client accepte une nouvelle variante sans modifier sa logique métier.

Pour aller plus loin · selon positionnement et temps disponible

Faire évoluer une famille d'objets sans propager des conditions partout.

Jour 2 · 7 heures

3. Composer les structures

210 minutes · apports, exercice et retour

- Utiliser Adapter pour une interface incompatible.
- Comparer Decorator, Facade et Composite selon le besoin.
- Éviter couches d'abstraction sans bénéfice mesurable.
- Distinguer Adapter, Decorator, Facade et Composite selon les relations recherchées.

NOTIONS ET MÉTHODES EN DÉTAIL

- Comparer Adapter, Facade, Decorator et Composite à partir de leurs collaborations.
- Distinguer adaptation de contrat et ajout de responsabilité autour d'un objet.
- Évaluer profondeur d'imbrication, transparence et coût de compréhension.

Atelier prévu

Brancher un fournisseur de prix externe via un adaptateur et conserver les tests du contrat interne.

Livrable attendu

Structure refactorisée et diagramme de dépendances.

Vérification en atelier

Le schéma montre précisément qui délègue à qui et pourquoi.

Pour aller plus loin · selon positionnement et temps disponible

Ajouter une capacité sans modifier le contrat attendu par les clients.

4. Faire varier les comportements

210 minutes · apports, exercice et retour

- Comparer Strategy, State et Template Method.
- Organiser notifications avec Observer et responsabilités explicites.
- Encapsuler une action avec Command si annulation ou journalisation l'exigent.
- Comparer Strategy, State, Observer et Command selon variabilité et temporalité.

NOTIONS ET MÉTHODES EN DÉTAIL

- Appliquer Strategy, State, Observer ou Command à un changement de comportement identifié.
- Distinguer variation d'algorithme et transition d'état métier.
- Prévoir ordre des notifications, erreurs et cycle de vie des abonnements.

Atelier prévu

Ajouter une stratégie de remise et une évolution d'état de commande sans empiler des conditions globales.

Livrable attendu

Comportements interchangeable et scénarios testés.

Vérification en atelier

Les transitions ou notifications sont testées sans dépendance à un ordre implicite.

Pour aller plus loin · selon positionnement et temps disponible

Traiter ordre des événements et effets secondaires dans un exemple concret.

Jour 3 · 7 heures

5. Vérifier les compromis

210 minutes · apports, exercice et retour

- Mesurer lisibilité, dépendances et coût d'extension.
- Tester par contrats et cas de comportement.
- Repérer surconception et abstraction prématurée.
- Évaluer lisibilité, extensibilité, allocation et complexité induite par les patterns.

NOTIONS ET MÉTHODES EN DÉTAIL

- Comparer solution directe et solution avec pattern sur lisibilité, extension et tests.
- Vérifier substitution et invariants des implémentations concurrentes.
- Repérer surconception et retirer une abstraction sans utilité démontrée.

Atelier prévu

Relire deux solutions au même besoin et supprimer un pattern qui complique inutilement l'exemple.

Livrable attendu

Matrice de compromis et décision documentée.

Vérification en atelier

L'argumentaire expose aussi le coût introduit par le pattern choisi.

Pour aller plus loin · selon positionnement et temps disponible

Retirer une abstraction devenue coûteuse sans casser le comportement.

6. Conduire un refactoring progressif

210 minutes · apports, exercice et retour

- Protéger le comportement avant la modification.
- Modifier une responsabilité à la fois.
- Documenter l'intention et les alternatives rejetées.
- S'appuyer sur tests de caractérisation, petites étapes et interfaces de transition.

NOTIONS ET MÉTHODES EN DÉTAIL

- Écrire tests de caractérisation avant de déplacer responsabilités et dépendances.
- Procéder par petites transformations conservant les interfaces utiles.
- Documenter décision, alternatives et point de variation effectivement obtenu.

Atelier prévu

Livrer le module refactoré et démontrer l'ajout d'une règle tarifaire avec un nombre limité de modifications.

Livrable attendu

Plan de refactoring et historique des vérifications.

Vérification en atelier

Le comportement reste stable et la nouvelle variante se branche au point prévu.

Pour aller plus loin · selon positionnement et temps disponible

Conduire une migration tout en conservant la compatibilité des appels existants.

Évaluation finale

Mise en situation individuelle et restitution commentée, incluses dans la dernière séquence. Durée indicative : 60 minutes.

- Chaque pattern répond à une variation identifiée.
- Les tests confirment la conservation du comportement.
- Le participant explique le coût de sa conception et une alternative plus simple.

Méthodes et ressources

Apports illustrés, exercices progressifs, ateliers sur un cas fil rouge et retour argumenté sur les productions. Les ateliers et l'évaluation font partie des heures indiquées.

Les ateliers décrivent les travaux prévus. Leurs fichiers, corrigés et évaluations en ligne sont à préparer, sauf ressources explicitement disponibles dans le module pilote.

Accès, accompagnement et contact

Pour obtenir plus de détails ou le programme détaillé validé, contactez le centre : 09 54 70 23 80 — contact@beyondexpertise.eu.

Positionnement et vérification des prérequis avant inscription. Délais, calendrier et aménagements à convenir avec le centre. Pour un besoin d'accessibilité, décrivez le besoin pratique sans diagnostic médical.

Programme enrichi proposé, à valider par le formateur avant animation. Les approfondissements sont adaptés au niveau et au temps disponible. Aucune certification professionnelle ni prise en charge CPF ou OPCO n'est garantie. Démarche Qualiopi en cours ; certification non acquise.

Références pour approfondir

- [OMG](#) · [spécification UML](#)